Name: Ankur De

Roll No: BL.SC.U4AIE24105

Assignment 6

Code

```
"""
Multimodal Gaussian + MRF MRI lesion segmentation.

Classes:
    0 = healthy / non-lesion brain tissue
    1 = diseased / lesion

Input:
    training/*mri_at1_t2space.nii
    training/*mri_at2.nii
    training/*t2_lesions.nii
    testing/*mri_at1_t2space.nii
    testing/*mri_at2.nii

The T1 and T2 images are combined into a 2-D feature vector per voxel.
A separate multivariate Gaussian is estimated for each class.
An MRF with a 6-connected Potts neighbourhood is then optimized using
red-black Iterated Conditional Modes (ICM).
"""

from pathlib import Path
import argparse
import re

import nibabel as nib
import numpy as np
```

```
TRAIN_DIR = "/run/media/ankur/Data/Ankur/Study/Semester 5/Probablistic Reasoning/Lab/Probablistic Reasoning/src/main/java/com/ankur/Lab_6/Data/training"

TEST_DIR = "/run/media/ankur/Data/Ankur/Study/Semester 5/Probablistic Reasoning/Lab/Probablistic Reasoning/src/main/java/com/ankur/Lab_6/Data/testing"

OUTPUT_DIR = "/run/media/ankur/Data/Ankur/Study/Semester 5/Probablistic Reasoning/Lab/Probablistic Reasoning/src/main/java/com/ankur/Lab_6/Outputs"


# Maximum number of voxels sampled from each subject/class when fitting.

# Lesion voxels are usually much fewer than healthy voxels.

MAX_SAMPLES_PER_CLASS_PER_SUBJECT = 100_000

RANDOM_SEED = 42


# Numerical regularization for the 2x2 covariance matrices.

COVARIANCE_REGULARIZATION = 1e-4


# MRF pairwise penalty. Increase it for smoother segmentations.

MRF_LAMBDA = 1000.0

MRF_ITERATIONS = 100


# Images in this dataset are scaled approximately from 0 to 100.

# Dividing by 100 keeps feature values in a convenient range.

INTENSITY_SCALE = 100.0


# A voxel is considered part of the brain if either modality has

# an intensity above this threshold after scaling.

BRAIN_THRESHOLD = 0.02



def subject_id_from_name(filename):

    """Return the subject identifier before the modality suffix."""

    for suffix in (

        ".mri_at1_t2space.nii",

        ".mri_at2.nii",

        ".t2_lesions.nii",
```

```
    ):
        if filename.endswith(suffix):
            return filename[:-len(suffix)]
    return None


def load_nifti(path):
    """
    Dataset-specific NIfTI loader.

    These files declare vox_offset=348 and store their voxel payload after
    the 348-byte header. MRI values are already approximately [0, 100] and
    lesion labels are already 0, 1, or 2. The supplied scl_slope/scl_inter
    fields are inconsistent with those stored values; applying them turns
    MRI intensities into a narrow range around 50 and breaks brain masking.

    Repair a temporary copy: insert four padding bytes, set vox_offset=352,
    and disable scaling (slope=1, intercept=0). Original files are untouched.
    """
    import os
    import struct
    import tempfile

    path = str(path)
    with open(path, "rb") as f:
        raw = f.read()

    if len(raw) < 348:
        raise ValueError(f"File is too small to be a NIfTI-1 image: {path}")

    little = struct.unpack("<i", raw[:4])[0]
    big = struct.unpack(">i", raw[:4])[0]
    if little == 348:
        endian = "<"
```

```
    elif big == 348:
        endian = ">"
    else:
        raise ValueError(f"Unsupported NIfTI-1 header: {path}")

    vox_offset = struct.unpack(endian + "f", raw[108:112])[0]

    if vox_offset < 352:
        print(f"[FIX] Repairing NIfTI header/scaling: {os.path.basename(path)}")
        repaired = bytearray(raw[:348] + b"\x00" * 4 + raw[348:])
        struct.pack_into(endian + "f", repaired, 108, 352.0)
        struct.pack_into(endian + "f", repaired, 112, 1.0)
        struct.pack_into(endian + "f", repaired, 116, 0.0)

        with tempfile.NamedTemporaryFile(suffix=".nii") as temp:
            temp.write(repaired)
            temp.flush()
            image = nib.load(temp.name)
            data = image.get_fdata(dtype=np.float32)
            detached_image = nib.Nifti1Image(
                data.copy(), image.affine.copy(), image.header.copy()
            )
        return detached_image, data

    image = nib.load(path)
    return image, image.get_fdata(dtype=np.float32)


def find_subject_files(directory, include_masks):
    """Pair T1, T2, and optionally lesion-mask files by subject ID."""
    subjects = {}

    for t1_path in directory.glob("*.mri_at1_t2space.nii"):
        sid = subject_id_from_name(t1_path.name)
```

```
        t2_path = directory / f"{sid}.mri_at2.nii"

        if not t2_path.exists():
            print(f"[WARNING] Missing T2 image for {sid}; skipping.")
            continue

        entry = {"t1": t1_path, "t2": t2_path}

        if include_masks:
            mask_path = directory / f"{sid}.t2_lesions.nii"
            if not mask_path.exists():
                print(f"[WARNING] Missing lesion mask for {sid}; skipping.")
                continue
            entry["mask"] = mask_path

        subjects[sid] = entry

    return subjects



def preprocess_modalities(t1, t2):
    """
    Scale both modalities and construct the brain mask.

    Both images are already registered to a common space in this dataset.
    Their shapes must match.
    """
    if t1.shape != t2.shape:
        raise ValueError(f"T1/T2 shape mismatch: {t1.shape} vs {t2.shape}")

    t1 = np.nan_to_num(t1 / INTENSITY_SCALE, nan=0.0,
                       posinf=0.0, neginf=0.0)
    t2 = np.nan_to_num(t2 / INTENSITY_SCALE, nan=0.0,
```

```
                    posinf=0.0, neginf=0.0)

    t1 = np.clip(t1, 0.0, 1.0)
    t2 = np.clip(t2, 0.0, 1.0)

    brain_mask = (t1 > BRAIN_THRESHOLD) | (t2 > BRAIN_THRESHOLD)

    # Shape: (number_of_voxels, 2)
    features = np.stack((t1, t2), axis=-1)

    return features, brain_mask

def sample_class_features(features, class_mask, max_samples, rng):
    """Randomly sample feature vectors belonging to one class."""
    indices = np.flatnonzero(class_mask.ravel())

    if len(indices) == 0:
        return np.empty((0, 2), dtype=np.float32)

    if len(indices) > max_samples:
        indices = rng.choice(indices, size=max_samples, replace=False)

    return features.reshape(-1, 2)[indices]


def fit_gaussian(samples):
    """
    Estimate a multivariate Gaussian:
        p(x | class) = N(x; mean, covariance)

    A small diagonal regularizer prevents singular covariance matrices.
    """
    if len(samples) < 2:
        raise ValueError("Not enough samples to estimate a Gaussian.")
```

```
    mean = samples.mean(axis=0)
    covariance = np.cov(samples, rowvar=False)
    covariance = covariance + COVARIANCE_REGULARIZATION * np.eye(2)

    sign, logdet = np.linalg.slogdet(covariance)
    if sign <= 0:
        raise ValueError("Covariance matrix is not positive definite.")

    inverse_covariance = np.linalg.inv(covariance)

    return {
        "mean": mean,
        "covariance": covariance,
        "inverse_covariance": inverse_covariance,
        "logdet": logdet,
    }


def gaussian_negative_log_likelihood(features, model):
    """
    Calculate -log p(x | class), ignoring the shared 2-D Gaussian constant.

    Lower values indicate a better match to the class distribution.
    """
    difference = features - model["mean"]

    mahalanobis = np.einsum(
        "...i,ij,...j->...",
        difference,
        model["inverse_covariance"],
        difference,
        optimize=True,
    )
```

```
    return 0.5 * (model["logdet"] + mahalanobis)


def train_models(training_subjects):
    """Train healthy and diseased multimodal Gaussian models."""
    rng = np.random.default_rng(RANDOM_SEED)

    healthy_samples = []
    diseased_samples = []

    print("\n========== TRAINING ==========")

    for sid, files in sorted(training_subjects.items()):
        print(f"\nLoading training subject: {sid}")

        _, t1 = load_nifti(files["t1"])
        _, t2 = load_nifti(files["t2"])
        _, lesion_mask = load_nifti(files["mask"])

        features, brain_mask = preprocess_modalities(t1, t2)

        if lesion_mask.shape != brain_mask.shape:
            raise ValueError(f"Mask shape mismatch for {sid}")

        # The lesion masks use NIfTI intensity scaling, and the scaling
        # factor differs between subjects. Therefore, a fixed threshold
        # such as 0.5 is incorrect. The non-zero label values are encoded
        # close to the mask maximum (subject 3 contains labels 1 and 2).
        mask_max = float(np.nanmax(lesion_mask))
        if not np.isfinite(mask_max) or mask_max <= 0:
            raise RuntimeError(
                f"Lesion mask for {sid} contains no positive labels. "
                f"Check the mask file and NIfTI scaling."
```

```
        )

    # Use half the maximum to separate background (approximately zero)
    # from all positive lesion labels, including labels 1 and 2.
    lesion_threshold = 0.5 * mask_max
    diseased_mask = (lesion_mask > lesion_threshold) &amp;amp; brain_mask
    healthy_mask = brain_mask &amp;amp; ~diseased_mask

    print(
        f"  Mask max: {mask_max:.8g}, "
        f"lesion threshold: {lesion_threshold:.8g}"
    )
    print(
        f"  Brain voxels: {np.count_nonzero(brain_mask):,}; "
        f"lesion voxels: {np.count_nonzero(diseased_mask):,}; "
        f"non-lesion brain voxels: {np.count_nonzero(healthy_mask):,}"
    )

    h = sample_class_features(
        features, healthy_mask,
        MAX_SAMPLES_PER_CLASS_PER_SUBJECT, rng
    )
    d = sample_class_features(
        features, diseased_mask,
        MAX_SAMPLES_PER_CLASS_PER_SUBJECT, rng
    )

    healthy_samples.append(h)
    diseased_samples.append(d)

    print(f"  Healthy voxels sampled:  {len(h):,}")
    print(f"  Diseased voxels sampled: {len(d):,}")

healthy_samples = np.concatenate(healthy_samples, axis=0)
```

```
    diseased_samples = np.concatenate(diseased_samples, axis=0)

    if len(healthy_samples) == 0 or len(diseased_samples) == 0:
        raise RuntimeError("One of the classes has no training samples.")

    print("\nFitting healthy Gaussian...")
    healthy_model = fit_gaussian(healthy_samples)

    print("Fitting diseased Gaussian...")
    diseased_model = fit_gaussian(diseased_samples)

    print("\nHealthy Gaussian mean [T1, T2]:")
    print(healthy_model["mean"])
    print("Healthy covariance:")
    print(healthy_model["covariance"])

    print("\nDiseased Gaussian mean [T1, T2]:")
    print(diseased_model["mean"])
    print("Diseased covariance:")
    print(diseased_model["covariance"])

    return healthy_model, diseased_model


def six_neighbour_sums(array, valid_mask):
    """
    Sum an array over its six axis-aligned neighbours.
    Neighbours outside valid_mask are ignored.
    """
    padded_values = np.pad(array, 1, mode="constant")
    padded_valid = np.pad(valid_mask, 1, mode="constant")

    centre = (
        slice(1, -1),
```

```
        slice(1, -1),
        slice(1, -1),
    )

    total = np.zeros(array.shape, dtype=np.float32)
    degree = np.zeros(array.shape, dtype=np.float32)

    for axis in range(3):
        for offset in (-1, 1):
            neighbour = [slice(1, -1)] * 3
            neighbour[axis] = (
                slice(0, -2) if offset == -1 else slice(2, None)
            )
            neighbour = tuple(neighbour)

            total += padded_values[neighbour] * padded_valid[neighbour]
            degree += padded_valid[neighbour]

    return total, degree


def mrf_icm(unary_healthy, unary_diseased, brain_mask,
            mrf_lambda=MRF_LAMBDA, iterations=MRF_ITERATIONS):
    """
    Minimize the binary MRF energy:

      E(L) = sum_p D_p(L_p)
           + lambda * sum_(p,q) [L_p != L_q]

    D_p is the Gaussian negative log-likelihood.
    The pairwise term is a Potts penalty over six-connected neighbours.

    Red-black updates allow non-adjacent voxels to be updated together.
    """
```

```
# Initial segmentation: choose the class with lower Gaussian cost.
labels = (unary_diseased < unary_healthy).astype(np.uint8)
labels[~brain_mask] = 0

# Checkerboard partition of the 3-D volume.
x, y, z = np.indices(labels.shape)
parity = (x + y + z) % 2

for iteration in range(iterations):
    changes = 0

    for colour in (0, 1):
        diseased_neighbour_count, degree = six_neighbour_sums(
            labels.astype(np.float32), brain_mask
        )

        # Cost of assigning healthy (0):
        # pay lambda for every diseased neighbour.
        cost_healthy = (
            unary_healthy
            + mrf_lambda * diseased_neighbour_count
        )

        # Cost of assigning diseased (1):
        # pay lambda for every healthy neighbour.
        cost_diseased = (
            unary_diseased
            + mrf_lambda * (degree - diseased_neighbour_count)
        )

        update_region = brain_mask &amp;amp; (parity == colour)

        new_values = (cost_diseased < cost_healthy).astype(np.uint8)
        changes += np.count_nonzero(
```

```
                labels[update_region] != new_values[update_region]
            )

            labels[update_region] = new_values[update_region]

        print(
            f"  MRF iteration {iteration + 1}/{iterations}: "
            f"{changes:,} voxel changes"
        )

        if changes == 0:
            break

    labels[~brain_mask] = 0
    return labels



def segment_test_subject(files, healthy_model, diseased_model):
    """Perform multimodal Gaussian classification followed by MRF."""
    sid = subject_id_from_name(files["t1"].name)

    print(f"\n========== TESTING: {sid} ==========")

    reference_image, t1 = load_nifti(files["t1"])
    _, t2 = load_nifti(files["t2"])

    features, brain_mask = preprocess_modalities(t1, t2)

    print("Calculating healthy Gaussian likelihood...")
    unary_healthy = gaussian_negative_log_likelihood(
        features, healthy_model
    )
```

```
    print("Calculating diseased Gaussian likelihood...")
    unary_diseased = gaussian_negative_log_likelihood(
        features, diseased_model
    )

    # Equal class priors are used to avoid the healthy class dominating
    # simply because it contains many more voxels.
    print("\nApplying MRF spatial regularization...")
    segmentation = mrf_icm(
        unary_healthy,
        unary_diseased,
        brain_mask,
    )

    return sid, reference_image, segmentation


def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--train", type=Path, default=TRAIN_DIR)
    parser.add_argument("--test", type=Path, default=TEST_DIR)
    parser.add_argument("--output", type=Path, default=OUTPUT_DIR)
    args = parser.parse_args()

    args.output.mkdir(parents=True, exist_ok=True)

    training_subjects = find_subject_files(args.train, include_masks=True)
    testing_subjects = find_subject_files(args.test, include_masks=False)

    if len(training_subjects) == 0:
        raise RuntimeError(f"No complete training subjects found in {args.train}")
    if len(testing_subjects) == 0:
        raise RuntimeError(f"No complete testing subjects found in {args.test}")
```

```
print(f"Training subjects: {len(training_subjects)}")
print(f"Testing subjects:  {len(testing_subjects)}")

healthy_model, diseased_model = train_models(training_subjects)

# Save the learned Gaussian parameters.
np.savez(
    args.output / "gaussian_models.npz",
    healthy_mean=healthy_model["mean"],
    healthy_covariance=healthy_model["covariance"],
    diseased_mean=diseased_model["mean"],
    diseased_covariance=diseased_model["covariance"],
)

for files in testing_subjects.values():
    sid, reference_image, segmentation = segment_test_subject(
        files, healthy_model, diseased_model
    )

    output_path = args.output / f"{sid}_lesion_segmentation.nii.gz"

    output_image = nib.Nifti1Image(
        segmentation.astype(np.uint8),
        reference_image.affine,
        reference_image.header,
    )
    output_image.set_data_dtype(np.uint8)
    nib.save(output_image, str(output_path))

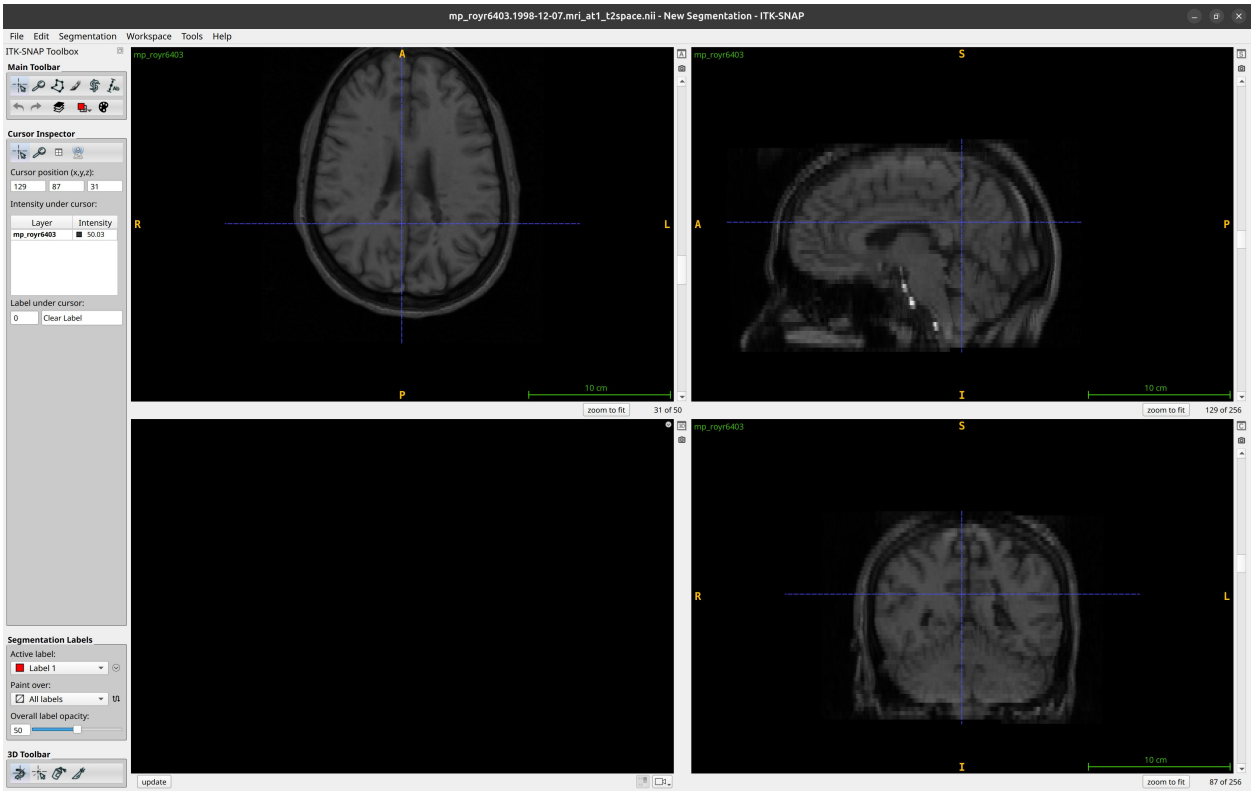
    lesion_voxels = int(np.count_nonzero(segmentation == 1))
    total_brain_voxels = int(np.count_nonzero(segmentation >= 0))

    print(f"\nSaved segmentation: {output_path}")
    print(f"Detected lesion voxels: {lesion_voxels:,}")
```

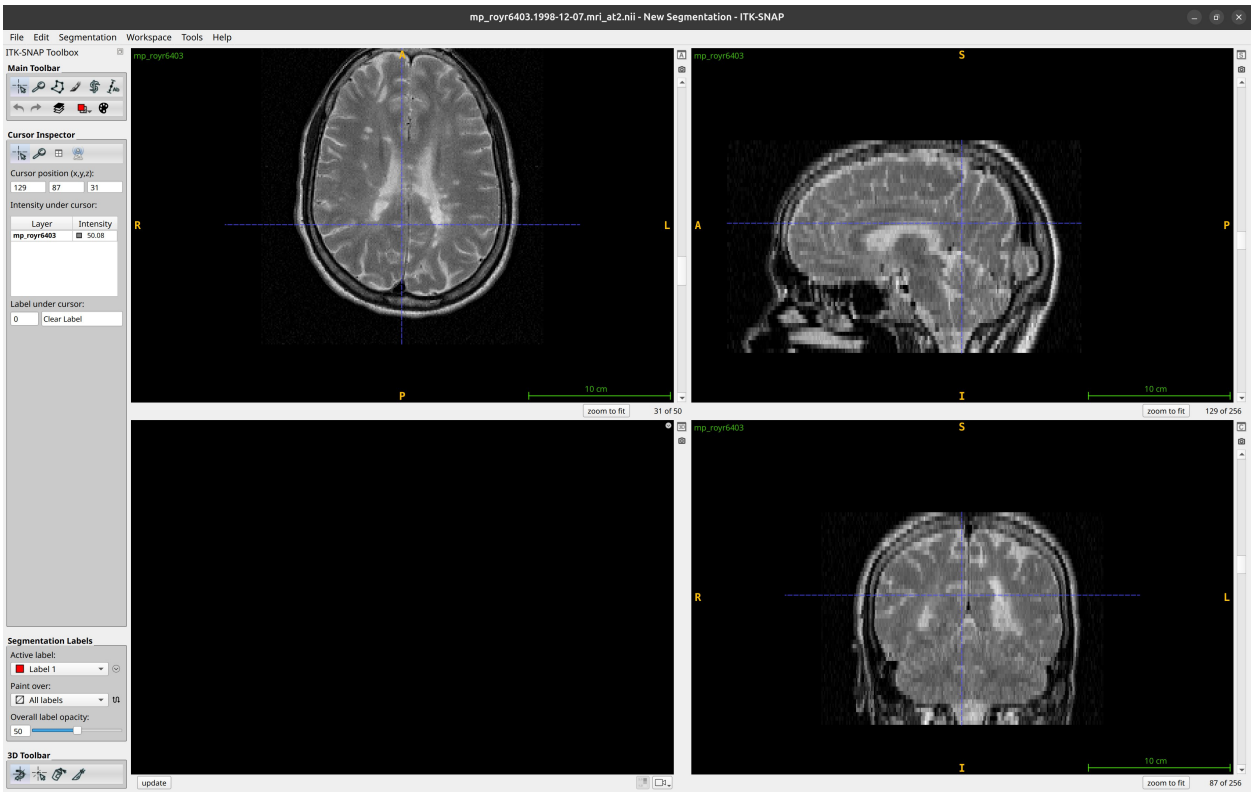
```
    print("\nProcessing completed.")
    print(f"Results are available in: {args.output.resolve()}")


if __name__ == "__main__":
    main()
```

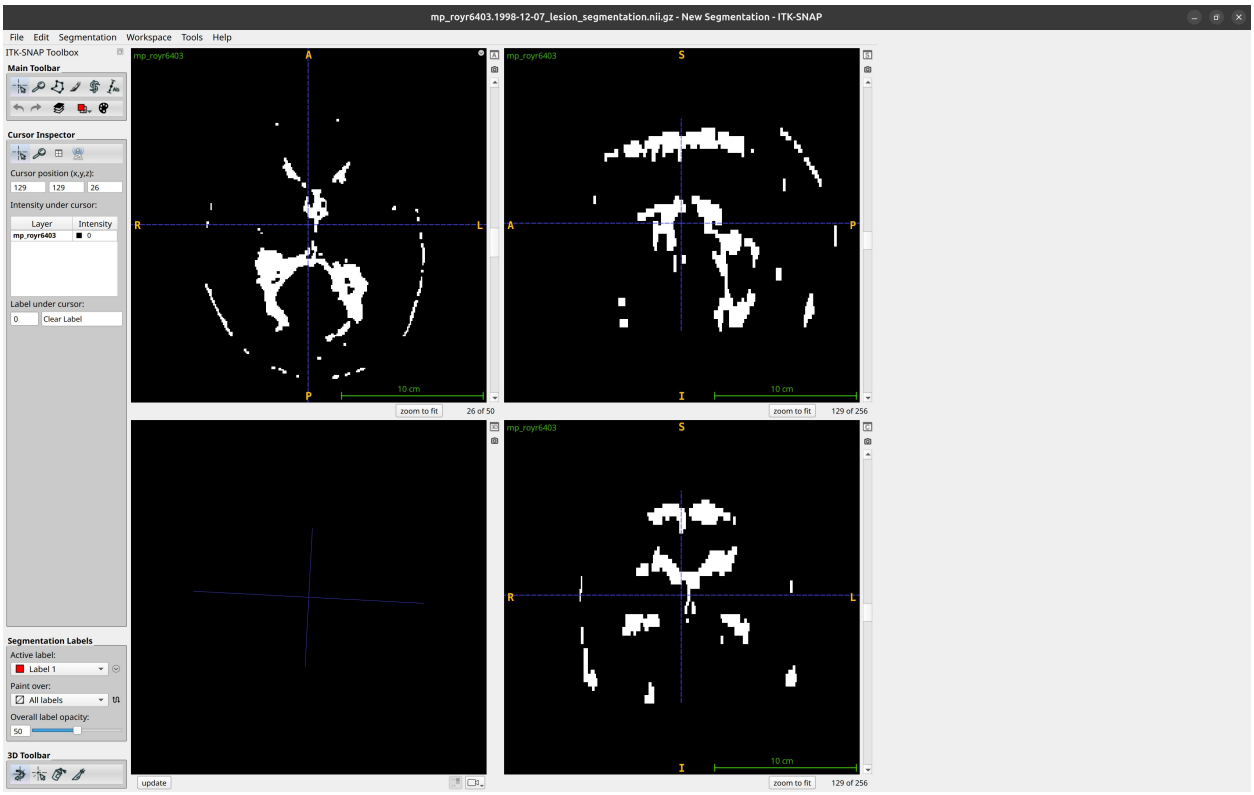
Results



T2 Space diagram


mp_royr6403.1998-12-07.mri_at2.nii - New Segmentation - ITK-SNAP
File Edit Segmentation Workspace Tools Help
ITK-SNAP Toolbox
Main Toolbar
Cursor Inspector
Cursor position (x,y,z):
129 87 31
Intensity under cursor:
Layer Intensity
mp_royr6403 50.08
Label under cursor:
0 Clear Label
Segmentation Labels
Active label:
Label 1
Paint over:
All labels
Overall label opacity:
50
3D Toolbar
mp_royr6403
A
R
L
P
10 cm
zoom to fit
31 of 50
mp_royr6403
S
A
P
I
10 cm
zoom to fit
129 of 256
update
mp_royr6403
S
R
L
I
10 cm
zoom to fit
87 of 256

A2 Space



Segmentation with MRF_LAMBDA = 1000.0, MRF_ITERATIONS = 100, BRAIN_THRESHOLD = 0.02